

ONEDRAFT

CAD-AI — Aria Powered

Frontend & User Interaction Architecture

Version 1.0

Document: OneDraft Frontend & User Interaction Architecture Specification

System: OneDraft CAD-AI

Intelligence Layer: Aria

Document Class: Architectural & Implementation Stack Specification

Status: Architectural Specification

Predecessor: Import, Export & External Data Exchange Specification

Successor: Project Lifecycle & State Machine Specification

1. PURPOSE

The Frontend & User Interaction Architecture establishes the complete user-facing application architecture of OneDraft.

The purpose of this subsystem is to provide the controlled interface through which users interact with projects, specifications, the computational Project Model, generated drawings, documentation, review workflows, external data, validation results and Aria intelligence.

The frontend shall not function as a disconnected presentation layer.

It shall operate as a controlled application interface over the OneDraft domain, application, orchestration, computational, documentation, validation, security and artifact systems.

The interface shall provide users with a coherent project workspace while preserving the distinction between user intent, system interpretation, authoritative project state, generated content and AI-generated proposals.

2. ARCHITECTURAL ROLE

The Frontend & User Interaction subsystem forms the primary human interaction boundary of OneDraft.

It connects users to the application services, Project Model, drawing system, review system, Aria intelligence, external-data system and operational services.

Primary responsibility: Provide a coherent, secure and deterministic user interface for controlling OneDraft project operations.

Primary inputs: User commands, project information, model state, drawings, validation results, artifacts, review records, system notifications and Aria responses.

Primary outputs: User commands, project changes, review actions, drawing interactions, redlines, configuration requests, workflow transitions and structured Aria interactions.

Authority: The frontend does not become the authoritative source of project truth. Authoritative state remains within the OneDraft backend and Project Model architecture.

Security boundary: Every operation initiated through the frontend shall be subject to the applicable identity, authorization and command-validation architecture.

3. FRONTEND ARCHITECTURE

The OneDraft frontend shall be implemented as a modular application capable of supporting the complete project lifecycle.

The architecture shall separate presentation, interaction state, application communication, project visualization and domain-specific user interfaces.

The frontend shall communicate with backend services through defined APIs and event channels rather than directly accessing persistence systems.

The interface shall support responsive interaction with both synchronous application operations and asynchronous jobs.

4. USER APPLICATION SHELL

The application shell shall provide the persistent framework surrounding individual project workflows.

The shell shall provide navigation, project identification, user identity, notifications, system status, workspace selection and contextual controls.

The shell shall remain consistent while the active project workspace changes.

The application shall clearly identify the active organization and project context so that user actions are not inadvertently performed against the wrong project.

5. PROJECT WORKSPACE

The Project Workspace shall provide the primary environment for interacting with an individual OneDraft project.

The workspace shall integrate project information, model state, drawings, documentation, validation, external information and Aria interaction.

The workspace shall provide contextual access to the current Project Model and project lifecycle state.

The interface shall make the distinction between draft, validated, reviewed, accepted and finalized information visible to the user.

6. PROJECT NAVIGATION

Project navigation shall provide structured access to the major project domains.

Navigation shall support access to:

Project Information: Project identity, requirements, site information and project metadata.

Project Model: Buildings, levels, grids, spaces, elements, assemblies and relationships.

Drawings: Views, sheets, plans, elevations, sections, details and schedules.

Validation: Compliance, coordination, geometry and system validation results.

Review: Redlines, comments, revisions, acceptance and approval workflows.

Artifacts: Generated files, imported files, exports and project packages.

External Data: Imported CAD, BIM, survey, PDF, image, specification and structured information.

Aria: AI interaction, proposals, commands, explanations and project assistance.

7. WORKSPACE MODES

The frontend shall provide context-specific workspace modes.

Workspace modes may include project overview, model, drawing, documentation, review, validation, external-data management and administrative configuration.

Each mode shall expose the information and actions appropriate to its context.

Mode transitions shall not discard unsaved or pending user state without explicit handling.

8. MODEL VIEWER

The Model Viewer shall provide interactive visualization of the computational Project Model.

The viewer shall support navigation, zooming, panning, rotation where applicable, object selection, visibility control and contextual inspection.

The viewer shall distinguish between model geometry and documentation graphics.

Users shall be able to select model objects and inspect their relevant properties, relationships, constraints and provenance.

9. MODEL OBJECT INSPECTION

Selecting an object shall provide access to its structured project information.

The interface shall present appropriate properties according to object type.

Object Identity: Unique OneDraft identifier.

Object Type: Domain classification.

Properties: Current parameter values.

Relationships: Related project objects.

Constraints: Applicable constraints.

Geometry: Associated geometry information.

Provenance: Origin and source information where applicable.

Revision State: Current model and revision context.

The frontend shall obtain authoritative object information from backend services.

10. DRAWING INTERFACE

The Drawing Interface shall provide controlled interaction with generated architectural documentation.

The interface shall support drawing viewing, sheet navigation, zooming, panning, annotation inspection, dimension inspection and review interactions.

Where editing is permitted, drawing modifications shall operate through structured documentation commands rather than uncontrolled graphical mutation.

The drawing interface shall preserve the distinction between model-derived graphics and documentation-specific content.

11. SHEET VIEWER

The Sheet Viewer shall display complete drawing sheets according to their defined sheet templates and graphic standards.

The viewer shall support sheet navigation, viewport inspection, drawing-scale awareness, title-block information and revision visibility.

The displayed sheet shall correspond to a defined drawing and sheet version.

Users shall be able to determine the source and revision context of the displayed artifact.

12. DRAWING VIEWPORTS

Drawing viewports shall provide controlled navigation into model-derived views.

Each viewport shall maintain its relationship to the underlying View object.

Viewport configuration shall include scale, orientation, visibility state and applicable graphic representation.

Changes to viewport configuration shall be recorded as structured documentation changes.

13. REDLINE INTERFACE

The frontend shall provide a dedicated redline interface.

Users shall be able to identify areas requiring correction, clarification, revision or further review.

Redlines shall remain structured review objects rather than becoming indistinguishable from authoritative drawing geometry.

Redline Identifier: Unique review identifier.

Location: Associated sheet, view or model context.

Author: User or process responsible for the redline.

Description: Structured review information.

Status: Current redline state.

Resolution: Response or corrective action associated with the redline.

14. ANNOTATION INTERACTION

Users shall be able to inspect and interact with supported drawing annotations.

Annotations shall retain their relationship to their source drawing, view or model object where applicable.

The frontend shall distinguish between generated annotations and manually created review annotations.

Editing an authoritative annotation shall require the appropriate command and authorization.

15. DIMENSION INTERACTION

Dimensions shall be selectable and inspectable within the drawing interface.

The interface shall identify whether a dimension is model-derived, documentation-derived or review-created.

Where dimensions are authoritative representations of model geometry, users shall be able to inspect the source references supporting the dimension.

Changes to model-derived dimensions shall occur through controlled model or documentation operations.

16. ARIA INTERFACE

The Aria Interface shall provide the primary natural-language interaction surface for OneDraft intelligence.

Users shall be able to communicate with Aria through structured conversational interaction within the project context.

Aria shall receive only the context authorized for the current user and project.

The interface shall distinguish between:

User Request: Original user instruction.

Aria Interpretation: Structured interpretation of the request.

Proposal: Proposed action or result.

Command: Deterministic executable operation.

Result: Outcome returned by the system.

Explanation: Human-readable description of the operation or result.

17. ARIA CONTEXT DISPLAY

The frontend shall provide appropriate visibility into the context being used by Aria.

Where applicable, the interface shall identify project, model, drawing, document or review context associated with an interaction.

Users shall not be required to assume that Aria has access to information that has not been supplied or authorized.

Context selection shall be explicit where ambiguity could materially affect an operation.

18. ARIA COMMAND CONFIRMATION

Commands capable of modifying project information shall receive appropriate user confirmation and deterministic validation according to their risk and authorization level.

Read-only requests may execute without additional confirmation where authorized.

Project-changing operations shall display sufficient information for the user to understand the proposed action.

The frontend shall not execute natural-language instructions directly against the database or Project Model.

19. COMMAND PREVIEW

Where a command produces meaningful project changes, the interface shall support a command preview.

The preview may identify affected objects, proposed changes, resulting values, affected drawings and validation implications.

Users shall be able to approve, reject or revise the proposed operation where the applicable workflow requires confirmation.

20. CHANGE VISUALIZATION

The frontend shall provide visual representation of meaningful project changes.

Changes may be displayed through model highlighting, drawing comparison, property comparison, revision overlays or other appropriate mechanisms.

The visualization shall distinguish proposed changes from committed changes.

21. REVISION COMPARISON

The interface shall support comparison between project, model and drawing revisions.

Comparison may include geometry, properties, annotations, dimensions, documentation and other supported project information.

The system shall identify additions, removals and modifications where technically possible.

Revision comparison shall use authoritative revision and change records.

22. VALIDATION INTERFACE

The frontend shall provide access to validation results.

Validation information shall be presented according to severity, category, affected object, affected drawing and resolution state.

Users shall be able to navigate from a validation result to the relevant project context where supported.

Severity: Classification of the validation result.

Status: Current validation state.

Affected Object: Project or documentation object associated with the result.

Rule: Validation rule that produced the result.

Evidence: Information supporting the result.

Resolution: Current corrective or disposition state.

23. COMPLIANCE INTERFACE

The frontend shall provide controlled access to regulatory and compliance information.

Users shall be able to inspect applicable requirements, validation outcomes and supporting project evidence where authorized.

Compliance results shall remain distinguishable from user commentary and AI recommendations.

The interface shall not represent an AI-generated interpretation as an authoritative regulatory determination.

24. PROJECT INFORMATION INTERFACE

The frontend shall provide structured interfaces for project requirements and project metadata.

Users shall be able to inspect and modify authorized project information through domain-specific forms and commands.

Fields shall correspond to structured domain properties rather than uncontrolled text wherever possible.

Changes shall pass through application services and create appropriate project revisions.

25. REQUIREMENTS INTERFACE

Project requirements shall be represented through structured interfaces.

Users shall be able to create, inspect, modify and organize requirements where authorized.

Requirements may include functional, spatial, regulatory, technical, performance and documentation requirements.

The interface shall preserve requirement identity and traceability throughout the project lifecycle.

26. EXTERNAL DATA INTERFACE

The frontend shall provide controlled interfaces for importing and managing external project information.

Users shall be able to initiate imports, inspect source artifacts, review extraction results, inspect validation conditions and approve supported incorporation operations.

External data shall remain visibly distinguishable from authoritative project information until incorporation has occurred.

27. IMPORT REVIEW

Imported information shall be reviewable before authoritative incorporation where the import workflow requires staging.

The interface shall display:

Source Artifact: Originating file or data source.

Detected Format: Interpreted source format.

Objects Found: Extracted or parsed objects.

Mapping: Proposed OneDraft classifications.

Conflicts: Detected inconsistencies.

Validation: Import validation results.

Incorporation State: Current state of the import.

The user shall be able to inspect and resolve supported conditions before incorporation.

28. EXPORT INTERFACE

The frontend shall provide controlled project export workflows.

Users shall select authorized export formats, profiles, project versions and applicable output content.

The interface shall present the resulting export state and artifact references.

Exports shall be registered through the artifact architecture.

29. ARTIFACT INTERFACE

The frontend shall provide access to project artifacts.

Artifacts may include generated drawings, CAD files, BIM files, PDFs, images, schedules, specifications, project packages and imported source files.

Users shall be able to inspect artifact metadata, provenance, revision and validation state according to their authorization.

30. JOB STATUS INTERFACE

Long-running operations shall provide visible job status.

The interface shall identify whether an operation is queued, running, awaiting validation, completed, failed or cancelled.

Where supported, users shall be able to inspect progress, affected artifacts and diagnostic information.

The frontend shall receive job updates through the application event architecture rather than continuously polling where an event-driven mechanism is available.

31. NOTIFICATION SYSTEM

The frontend shall provide a notification system for meaningful project and system events.

Notifications may include completed jobs, validation failures, review requests, project changes, import results, export completion and system warnings.

Notifications shall be associated with their originating project or system context.

32. SEARCH

OneDraft shall provide project-aware search.

Search shall support project objects, drawings, sheets, requirements, artifacts, reviews, validation results and other searchable domain information.

Search results shall preserve object identity and provide navigation into the relevant workspace context.

Search shall respect authorization boundaries.

33. FILTERING AND SORTING

Domain interfaces shall provide filtering and sorting appropriate to the displayed information.

Filters shall operate against structured domain properties.

User-selected filters may be persisted as workspace preferences where appropriate.

Filtering shall not modify underlying project information.

34. USER STATE

The frontend shall maintain controlled user-interface state.

UI state may include active project, selected objects, active sheet, drawing zoom, filters, panel state, navigation state and temporary form information.

UI state shall remain separate from authoritative project state.

Temporary state shall not become project data until an authorized application operation commits it.

35. UNSAVED CHANGES

The frontend shall identify pending changes before they are committed.

Forms and interactive editors shall provide clear indication when local changes have not been persisted.

Navigation away from pending changes shall trigger appropriate preservation, confirmation or cancellation behavior.

The frontend shall not silently discard meaningful user input.

36. REAL-TIME APPLICATION STATE

Where appropriate, the frontend shall receive real-time state changes through the OneDraft event architecture.

Real-time updates may include job progress, project changes, review activity, validation results and system notifications.

The frontend shall reconcile incoming state changes without corrupting active user interaction.

37. COLLABORATION

The frontend shall support controlled multi-user project interaction.

Where multiple authorized users operate on the same project, the interface shall identify relevant changes and revision states.

Concurrent modification conflicts shall be handled through backend-controlled revision and change mechanisms.

The frontend shall not assume that the local state is the current authoritative project state.

38. ACCESS CONTROL IN THE FRONTEND

Frontend controls shall reflect the user's authorized capabilities.

Restricted actions shall not be presented as executable where the user's permissions do not permit them, while backend authorization shall remain the ultimate enforcement boundary.

The interface shall distinguish between unavailable actions, unauthorized actions and actions that are temporarily unavailable because of project state.

39. ACCESSIBLE USER INTERACTION

The frontend shall be designed to support accessible interaction.

Controls shall have meaningful labels and predictable behavior.

Keyboard navigation shall be supported where practical.

Interactive elements shall expose appropriate semantic information to assistive technologies.

Visual indicators shall not be the sole mechanism for communicating critical project state.

40. RESPONSIVE APPLICATION DESIGN

The frontend shall support appropriate operation across supported desktop and tablet environments.

The architectural drafting and model-viewing interfaces shall prioritize professional workstation use while maintaining adaptable application navigation.

Responsive behavior shall preserve project context and prevent accidental destructive actions.

41. FRONTEND COMPONENT ARCHITECTURE

The frontend shall use reusable components organized around application and domain responsibilities.

Representative component categories include:

Application Shell: Global navigation and system context.

Project Workspace: Project-level composition.

Model Viewer: Computational model visualization.

Drawing Viewer: Drawing and sheet visualization.

Property Inspector: Object and property inspection.

Validation Panel: Validation and compliance results.

Review Panel: Redlines, comments and acceptance.

Aria Interface: AI interaction and command presentation.

Artifact Browser: Project artifact management.

Import Manager: External-data ingestion.

Export Manager: Controlled project export.

Notification Center: System and project notifications.

Search Interface: Project-aware search.

42. FRONTEND STATE MANAGEMENT

Frontend state management shall distinguish among local UI state, server state, project state and real-time event state.

UI State: Temporary presentation and interaction state.

Server State: Data retrieved from backend services.

Project State: Authoritative domain state.

Event State: Newly received system changes requiring reconciliation.

The frontend shall avoid duplicating authoritative business logic inside client-side state management.

43. API COMMUNICATION

All authoritative application communication shall use defined API contracts.

The frontend shall use typed request and response models where supported.

API errors shall be represented through structured error handling.

The frontend shall not infer successful state changes merely because an HTTP request was sent.

Successful project mutations shall be confirmed by authoritative backend responses.

44. ERROR HANDLING

Frontend errors shall be presented according to their operational significance.

Errors shall distinguish between validation errors, authorization errors, unavailable services, network failures, conflicts, unsupported operations and unexpected system failures.

Where recovery is possible, the interface shall provide an appropriate recovery path.

Error messages shall provide sufficient information for users to understand the operational result without exposing sensitive implementation details.

45. SECURITY AND SESSION MANAGEMENT

The frontend shall operate within the identity and access architecture.

Authentication state shall be managed securely.

Session expiration shall be handled explicitly.

Sensitive information shall not be stored in uncontrolled client-side storage.

Authorization shall remain enforced by backend services even when frontend controls are hidden or disabled.

46. OBSERVABILITY

Frontend operations shall contribute appropriate telemetry to the OneDraft observability architecture.

Telemetry may include application errors, API failures, performance metrics, interaction latency, job-state transitions and frontend availability.

Telemetry shall avoid unnecessarily collecting sensitive project information.

Operational identifiers shall permit frontend failures to be correlated with backend operations where appropriate.

47. TESTING

The frontend shall be subject to unit, component, integration, contract, end-to-end, accessibility, visual regression and workflow testing.

Tests shall verify:

Navigation: Project and application navigation.

Interaction: Forms, controls and workflows.

API Contracts: Correct communication with backend services.

Model Interaction: Selection and inspection of project objects.

Drawing Interaction: Sheets, views, annotations and redlines.

Aria Interaction: Context, proposals and command workflows.

Review: Redlines, comments and acceptance.

Security: Authorization-aware user interaction.

Error Handling: Correct response to application failures.

48. PERFORMANCE

The frontend shall provide responsive interaction for normal project operations.

Large models and drawing sets shall use appropriate lazy loading, virtualization, caching and incremental rendering strategies.

Heavy computational operations shall remain server-side or worker-side rather than blocking the user interface.

Model and drawing visualization shall be optimized for the expected OneDraft project scale.

49. ARCHITECTURAL INVARIANTS

The following invariants shall govern the Frontend & User Interaction Architecture.

Backend Authority: The frontend shall not become the authoritative source of project truth.

Structured Interaction: User actions shall resolve into structured application operations.

Security Enforcement: Backend authorization shall remain authoritative.

Project Context: User actions shall remain associated with an explicit organization and project context.

AI Separation: Aria interpretation and proposals shall remain distinguishable from authoritative project state.

Deterministic Commands: Project-changing AI actions shall pass through deterministic command validation.

State Separation: Temporary UI state shall remain separate from authoritative project state.

No Silent Loss: Meaningful unsaved user input shall not be silently discarded.

Revision Integrity: User-visible project changes shall correspond to authoritative revision and change records.

External Data Separation: Imported information shall remain distinguishable from authoritative project information until incorporated.

Auditability: Meaningful project-changing actions shall remain traceable.

Consistency: The interface shall present project state according to authoritative backend information.

50. COMPLETION STATUS

The Frontend & User Interaction Architecture establishes the complete human interaction layer of OneDraft.

The subsystem provides the application shell, project workspace, model viewer, drawing interface, sheet viewer, review and redline interfaces, validation and compliance interfaces, external-data workflows, artifact management, job status, notifications, search, collaboration, access-control presentation and Aria interaction.

The frontend architecture establishes the controlled boundary through which users operate the OneDraft platform while preserving backend authority, Project Model integrity, security enforcement, provenance and deterministic command execution.

The next architectural layer establishes the formal Project Lifecycle & State Machine through which these user interactions transition projects between defined operational states from intake through specification, modeling, validation, documentation, review and acceptance.

Status: ARCHITECTURALLY DEFINED

Next Document: Project Lifecycle & State Machine Specification

Overall Architecture: OneDraft End-to-End System Integration